How tokens work in your authController.js ?

🔑 Types of Tokens

This controller uses **two tokens**:

1. **Access Token**
   - Short-lived (usually minutes).
   - Used to authenticate API requests (e.g., calling a protected route).
   - Stored in **memory (frontend app)**, not in cookies.
   - Signed using signAccessToken(...).
2. **Refresh Token**
   - Long-lived (usually days or weeks).
   - Stored **in a secure, HTTP-only cookie** on the client (cannot be accessed by JavaScript).
   - Used to request new access tokens when the old one expires.
   - Signed using signRefreshToken(...).
   - Stored **hashed** in the database (user.refreshTokens array).
     - The actual token never gets stored directly, only its hash.

⚙️ Token Flow in Each Function

1. Register

- A new user signs up.
- Password is hashed with **bcrypt** → passwordHash.
- After saving user:
  - An **access token** is generated → returned in response.
  - A **refresh token** is generated:
    - Saved as a **hash** in DB → user.refreshTokens.push({ token: hashToken(refreshToken) }).
    - The real refresh token is sent to the client in a **secure cookie**.
- **Result**: User gets logged in right after signup.

2. Login

- User provides email + password.
- System verifies password using bcrypt.compare().
- On success:
  - Creates **access token** (for immediate API use).
  - Creates **refresh token** (for renewing access tokens later).
    - Hash stored in DB.
    - Real refresh token stored in HTTP-only cookie.
- **Result**: User is logged in with fresh tokens.

3. Refresh

- Triggered when access token **expires**.
- Steps:
  1. Backend checks req.cookies.refreshToken.

2. Validates it with verifyRefreshToken().
3. Finds user in DB.
4. Compares the **hash** of the cookie's refresh token against the DB's stored tokens.
5. If valid:
    - Removes the old refresh token (rotation: prevent replay attacks).
    - Issues a **new refresh token** (stored in DB + cookie).
    - Issues a **new access token** (sent in JSON response).

- **Result**: User stays logged in without re-entering password.

4. Logout

- User sends a logout request.
- Backend checks the refresh token cookie.
- If valid, removes its **hashed version** from DB.
- Clears the cookie.
- **Result**: Refresh token is invalidated → user must log in again.

🔒 Security Measures

1. **Hashing Refresh Tokens**
    - Just like passwords, refresh tokens are hashed before storing in DB.
    - Prevents token leaks if DB is compromised.
2. **Rotation**
    - Old refresh tokens are deleted whenever a new one is issued.
    - Prevents reuse of stolen tokens.
3. **HTTP-only Cookies**
    - Refresh token is stored in cookie with:
        - httpOnly: true → JS on client cannot access it.
        - secure: true (if HTTPS) → only sent over HTTPS.
        - sameSite: Strict → prevents CSRF.
4. **Short-lived Access Tokens**
    - Even if stolen, attacker only has limited time before it expires.

🌀 Example Flow

1. **Register/Login**
    - Client gets:
        - accessToken (in response JSON)
        - refreshToken (in secure cookie)
2. **Call API**
    - Client attaches accessToken in Authorization: Bearer <token> header.
3. **Access Token Expires**
    - Client automatically sends a request to /auth/refresh.
    - Backend checks refresh token cookie → issues new accessToken + refreshToken.
4. **Logout**
    - Client sends logout → backend removes refresh token from DB + clears cookie.

👉 So in simple terms:

- **Access Token = Short-lived ID card** 🪪 (use it to enter API "rooms").
- **Refresh Token = Long-lived key** 🔑 (used to get a new ID card when old one expires).
- Both work together to keep the session secure and smooth.

FlowChart: